JAVA PROGRAMMING FUNDAMENTALS

Java Encapsulation & Inheritance

WHAT IS ENCAPSULATION?

It protects the data inside the class by allowing access only through methods

- Protects data from invalid changes
- Improves data integrity
- Makes the code easier to maintain

Data Hiding: Protecting State

⚠Direct Access (Unsafe)

Allowing external code to touch fields directly makes data vulnerable to corruption or invalid values.

```
// Vulnerable direct access
student.name = "RJ";
student.age = -99; // Invalid age!
```

Anyone can set illegal values without restriction or validation!

✅Controlled Access (Safe)

Internal data is kept hidden. External code interacts strictly through getter and setter methods.

```
// Controlled setter method
student.setName("RJ");
student.setAge(20); // Validated!
```

Setters intercept changes, verifying data integrity beforehand.

Real-World Analogy: ATM Machine

Encapsulation in Daily Life

Imagine standing in front of a bank's ATM machine. You need cash, but you cannot open the machine vault directly to grab money.

- **Protected Data:** Your bank account balance in the database.
- **Controlled Interface (Methods):**
  - `withdraw(amount)`
  - `deposit(amount)`
  - `checkBalance()`

> The ATM verifies your PIN and checks for sufficient funds before balance adjustments occur. Java encapsulation works the exact same way!

What is Data Validation?

Data validation is the process of checking whether the user's input or data is correct, acceptable, and within the allowed range before storing it in a variable.

- **Prevents Invalid Data**
  Ensures that incorrect values (e.g., negative age or grade above 100) are not stored in the program.
- **Improves Data Accuracy**
  Accepts only valid and meaningful information, making the program's data more reliable.
- **Reduces Program Errors**
  Helps prevent bugs, crashes, or incorrect calculations caused by invalid user input.

Java Access Modifiers

Access Modifier	Same Class	Same Package	Subclass (Diff Pkg)	Everywhere
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Gold Standard Practice: Instance variables (attributes) should be , while public interaction methods are . `public` `private`

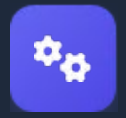
Core Benefits of Encapsulation

Data Security

Hides critical attributes from external unauthorized modifications, preventing unintended side effects or system corruption.

Validation

Enforces business rules inside setter methods to guarantee objects always maintain a valid and consistent state.



Maintainability

Allows developers to alter internal class logic seamlessly without breaking existing external consumer code.

Getters and Setters

Getter (Accessor)

Provides **read-only** access to a private instance variable.

```
public String getName() {
    return this.name;
}
```

Naming Convention: Prefix with + FieldName.

`get`

Setter (Mutator)

Provides **controlled write** access to update variable values.

```
public void setName(String name) {
    this.name = name;
}
```

The keyword resolves name collision with method arguments.

`this`

Encapsulated Student Class

```
public class Student {

        private String name; // Private instance variables
        private int age; // Constructor initializing fields via setters

public Student(String name, int age) {
        this.name = name; setAge(age); // Leverage validation logic
}

 public String getName() {
        return name;
 }

 public void setName(String name) {
        this.name = name;
}
public int getAge() {
        return age;
}
public void setAge(int age) {
        if (age >= 0 && age <= 120) {
                this.age = age;
} else {
        System.out.println("Error: Invalid age!");
}
 }
 }
```

Data Validation & Integrity

100%

Data Integrity Control

Zero corrupted or invalid states allowed into memory.

Setters as Gatekeepers

Because fields are private, external code cannot force improper states. Setter methods act as guards that audit data input.

```
public void setAge(int age) {
    // Validation business logic
    if (age ≥ 0 && age ≤ 120) {
        this.age = age;
    } else {
        // Rejects bad input cleanly
        System.out.println("Invalid age parameter.");
    }
}
```

Refactoring: Before vs After

✗Before (Poor Design)

```
class Student {
    public String name; // Public field!
    public int age;
}

// Direct mutation elsewhere
Student s = new Student();
s.age = -10; // Allowed! Broken state!
```

✓After (Encapsulated)

```
class Student {
    private String name;
    private int age;
    // Getters, setters & validation
}

Student s = new Student();
// s.age = -10; // Compile Error!
s.setAge(-10); // Caught by Setter!
```

What is for loop?

A for loop is a control structure used to repeat a block of code a specific number of times.

- **Reduces Repetitive Code**
  Executes the same block of code multiple times without rewriting it.
- **Saves Time and Improves Efficiency**
  Performs repetitive tasks automatically, making programs shorter and faster to write.
- **Makes Code Easier to Read and Maintain**
  Keeps programs organized and easier to update compared to writing the same statements repeatedly.

Introduction to the for Loop

The loop executes a block of code repeatedly a specific, predetermined numbe`for`imes.

```
for (initialization; condition; update) {
    // Code to execute repeatedly
}
```

1. Initialization

Runs once at

`int i = 0`

2. Condition

Checked before

`i < 5`

3. Update

Executes after loop

`i++`

Using for Loops with Objects

Combining loops with object arrays or collections allows automated creation and encapsulated reading.

for

```
Student[] students = new Student[3]; // Array of 3 Students

// Populating array using loop
for (int i = 0; i < students.length; i++) {
    students[i] = new Student("Student " + (i + 1), 18 + i);
}

// Displaying student info using getter methods
for (int i = 0; i < students.length; i++) {
    System.out.println(students[i].getName() + " - Age: " + students[i].getAge());
}
```

Scenario

A school library wants to create a Library Book Registration System to record newly acquired books. The system should securely store each book's information, accept only valid data, and display all registered books after the registration process is complete. Design the system using Object-Oriented Programming principles and use a for loop to process multiple book records.

WHAT IS INHERITANCE?

-"Reuse , Don't Repeat"
-Inheritance allows a class (child) to inherit the attributes and methods of another class (parent)

- REUSE EXISTING CODE
- REDUCE REDUNDANCY
- EASY TO MAINTAIN AND EXTEND

What is Switch case?

A switch case is a control statement in many programming languages that lets you execute different blocks of code based on the value of a single expression

- **Makes code easier to read**
- It is clearer when choosing between multiple fixed options.
- **Handles multiple choices efficiently**
- It is useful when one variable can have several possible values, such as menu choices.
- **Reduces long if-else if statements**
- Instead of writing many conditions, you can organize the choices using case.

A restaurant plans to develop a Food Registration and Ordering System to manage its menu items. The system should securely store each food item's information using at least five (5) attributes, support different types of food through inheritance, and ensure that only valid information is accepted before it is stored. Each child class must contain at least one (1) unique attribute that is not included in the parent class.The program must use Scanner for user input and implement a menu-driven interface using a while loop and a switch statement with the following menu options: Register Food, Display All Registered Foods, Exit. Use a for loop to register and display multiple food records. Apply Object-Oriented Programming principles, including encapsulation, inheritance, and the super keyword, to

Assignment # 1

1. What is Encapsulation?
2. What is Data Hiding?
3. What is an Access Modifier?
4. What is a Getter?
5. What is a Setter?

Assignment # 2

1. What is Data Validation?
2. Why is Data Validation important? Give at least three (3) reasons.
3. What is a for loop?
4. Explain the difference between this.variable = value; and setVariable(value);.
5. Why is Encapsulation important in Object-Oriented Programming? Give at least three (3) reasons.

Activity # 1

A cinema plans to develop a Movie Ticket Reservation System to manage customer reservations efficiently. The system should securely store each reservation using at least four (4) attributes, ensure that only valid information is accepted, and display all reservations after the registration process is complete. The system should be designed following the principles of Object-Oriented Programming, with a focus on protecting and organizing data effectively.

Activity # 2

A school wants to create a simple Java program that allows students to register for different school clubs. There are different types of clubs, and each type may have information that is common to all clubs as well as information specific to that type of club.

The program should allow the user to repeatedly choose an option from a menu until they decide to exit.

Instructions:
Use switch-case, for loop, inheritance, and encapsulation.

ENCAPSULATION

"Protect Your Data, Control the Access."

What is Encapsulation?

It protects the data inside a class by allowing access only through methods.

Why Encapsulation?

- Protects data from invalid changes
- Improves data integrity
- Makes code easier to maintain

Example: Student Information System

We don't want anyone to change the GPA directly. It must be updated only if the value is valid.

Student
- studentNo - name - gpa (private)
+ getName() + setName(String name) + getGPA() + setGPA(double gpa)

How to access?

✗ student.gpa = 5.0

✓ student.setGPA(1.75) (proper way)

Data is modified through methods with validation.

```
class Student {
    private double gpa;

    public double getGPA() {
        return gpa;
    }

    public void setGPA(double gpa) {
        if (gpa >= 1.0 && gpa <= 5.0) {
            this.gpa = gpa;  // valid
        } else {
            System.out.println("Invalid GPA!");
        }
    }
}
```

← Allows controlled reading of data.

← Allows controlled updating of data with validation.

SIR LD
IT Learning & Growth
— LD TechLab —

INHERITANCE

"Inheritance = Reuse, Don't Repeat."

What is Inheritance?

Inheritance allows a class (child) to inherit the attributes and methods of another class (parent).

Example: Student Information System

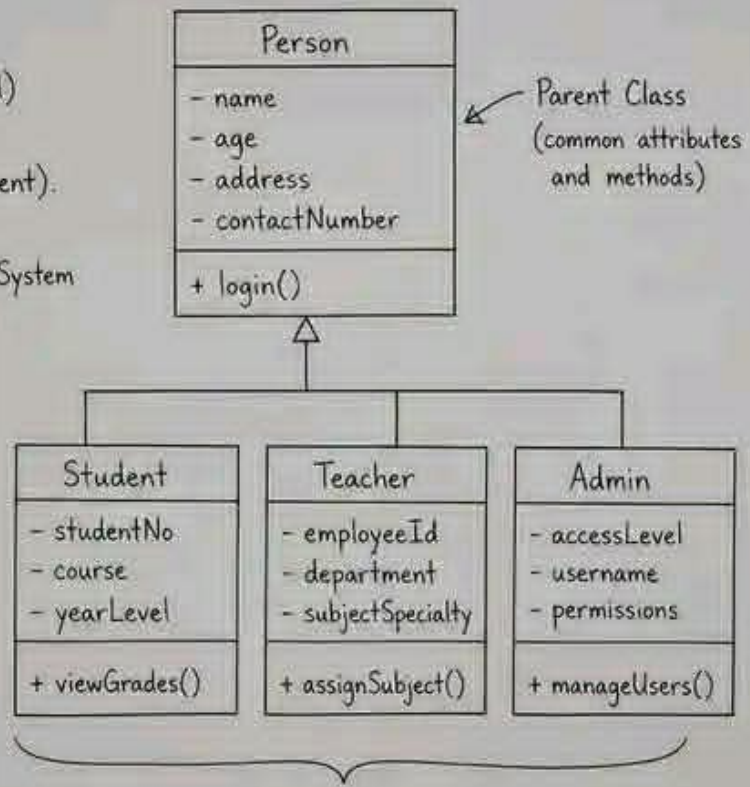
Lahat ng users sa system ay may common information. Kaya gagawa tayo ng isang parent class na Person.

Benefits:

- Reuse existing code
- Reduce redundancy
- Easy to maintain and extend

In Java (example):

```
class Student extends Person {
    // additional attributes and methods
}
```



SIR LD
IT Learning & Growth
— LD TechLab —